DCC071 Theory and Practice of SMT Solving
2026.2

Course project proposal

Team members

- Lara Amélia Maia de Freitas (2024005912);
- Jorge Augusto de Lima e Silva (206688138).

1 Problem and Motivation

Most SMT solvers decide quantifier-free linear integer arithmetic (QF_LIA) by solving the real relaxation with a dual simplex [1] and then using branch-and-bound and cuts [2], [3] until they find an integer point. Simplex returns vertices, so this search stays on the *surface* of the polyhedron and can be very slow on problems that have many integer solutions, such as unbounded polyhedra with infinite lattice width.

Bromberger and Weidenbach [4] proposed the *unit cube test*, which looks at the *interior* instead. A polyhedron $Ax \leq b$ contains an axis-parallel cube of edge length 1, and hence an integer point, iff the shifted system $Az \leq b'$ with $b'_i = b_i - \frac{1}{2}\|a_i\|$ has a real solution z. Rounding z then gives an integer solution of $Ax \leq b$. Therefore, if the test succeeds, we can get a valid integer solution from a single LP call (one extra simplex call), producing a model that can be checked with no branching (as in such cases, we would not need to call and use a Branch-And-Bound solver). In their solver SPASS-IQ, it solved all 1,209 instances of their evaluation (980 with infinite lattice width from four SMT-LIB families, plus 229 derived bounded ones), about 25× faster than without it. CVC4, MathSAT, Yices and Z3 (2016 versions) each left 52–178 of them unsolved within 10 minutes. If the constraints contain or *imply* an equality, the polyhedron has no interior and the test always fails. The paper sketches a "correction" to these cases [4, Sec. 6] such that the test can still be applied: detect implied equalities by checking whether the strict system $Ax < b$ is satisfiable, read them off the conflict explanation, and eliminate them with a Diophantine equation solver [3].

Gap and contribution Z3 [5] already includes a cube test; cvc5 [6] does not. Its default `--arith-brab` option, which its documentation compares to a unit cube test, rounds *one* variable at a time to its nearest integer n and adds the lemma $x{=}n \vee x{\leq}n{-}1 \vee x{\geq}n{+}1$ with $x{=}n$ preferred. Nothing guarantees that this rounding stays feasible. The cube test rounds *all* variables at once, from a point that provably rounds to a solution. OpenSMT has no cube test either. Our contribution is, therefore, its implementation and evaluation inside cvc5, a CDCL(T) solver that already has a Diophantine equation solver, branch-and-bound and brab. That way, in problems where the test works (and where it could succeed due to not having equalities or implied equalities in the set of constraints), we could avoid the need to perform expensive branch-and-bound calls for finding solutions, and check how that affects its effectiveness (in terms of problems it is able to solve) and efficiency.

Research questions **RQ1:** Does the unit cube test increase the number of QF_LIA instances cvc5 solves, and what does it cost where it does not help? **RQ2:** Does eliminating implied equalities make the test succeed where it otherwise fails? **RQ3:** When should the test be called, and how does it interact with brab?

2 What we will build

We will extend the linear arithmetic solver of cvc5 1.4.1, the latest release, forking at its tag. We chose cvc5 over OpenSMT because it already has strict bounds via δ-rationals, conflict explanations from simplex and a Diophantine equation solver [3]. We restrict ourselves to pure integer problems. Everything will sit behind new options and report statistics (calls, successes, time).

C1 Unit cube test. The test runs at full effort when the real relaxation is satisfiable but its model is not integral, after the Diophantine solver and right before cvc5 branches (in `TheoryArithPrivate::postCheck`). This follows the paper's advice to check the real relaxation first. We shift each row bound by $\frac{1}{2}\|a_i\|$ and each variable bound by $\frac{1}{2}$, solve, round, and check the rounded point against all asserted constraints, including disequalities. A first-week spike will settle two design choices. **(a)** Where to solve the shifted system: in place, under a scoped context push as cvc5 already does for its branch-cut replay, or in a separate exact simplex instance, which C3 needs anyway because it changes the variables. **(b)** How a successful point becomes the model: install it directly, which must respect the tableau invariants, or emit brab's own lemma for every variable with the rounded value as the preferred phase. Those lemmas are tautologies, so the second option is sound by construction.

C2 Implied-equality detection. When the shifted system is infeasible, we check $Ax < b$. If it is infeasible, every constraint in the explanation holds with equality [4, Lemma 6]. We repeat until $Ax < b$ is satisfiable.

C3 Equality elimination. We substitute the equalities away to get an equisatisfiable system over fewer variables that has an interior, run C1 on it, and map the integer point back. We will try to reuse cvc5's Diophantine solver first. Its public interface only exposes substitutions that introduce no new variables, so if we cannot reach the full solved form $x = x_0 + Ty$ we will write our own Hermite-normal-form elimination.

Soundness by design. C1–C3 never answer *unsat*. A conflict derived from shifted or strict bounds is not a valid theory conflict, because those bounds are not asserted facts, so it is discarded and never reaches the SAT solver. If no integer point is found, the components give up and cvc5 continues as usual. The only thing they produce is an integer point, checked against the original constraints.

3 Evaluation

Benchmarks Every set will be reconstructible by a script. (i) The paper's SMT-LIB QF_LIA families [7] (CAV-2009, dillig, slacks, prime-cone) and its *rotate* family, regenerated from its definition. (ii) Bromberger's unbounded set (`20180326-Bromberger`). (iii) RQ2 targets: variants of (i) whose equalities are implied, have non-unit coefficients, or sit under Boolean structure. We add the SMT-LIB instances on which C1 fails because the shifted system is infeasible, reported separately because our own tool selects them. (iv) 1,000 other QF_LIA instances drawn with a fixed random seed, to measure overhead. Other benchmarks may be added as found in related work papers.

Configurations Comparisons will be made between different cvc5 configurations, with the main goal of comparing how they behave with and without the test. Configs. are as follows: B0: cvc5 1.4.1 default (brab on); B0$^-$: brab off; B1 and B1$^-$: B0 and B0$^-$ plus C1; B2: B0 plus C1–C3. External references: Z3, which has its own cube test, and SPASS-SATT, the authors' solver, if its binary still runs.

Measures and budget Solved instances and PAR-2, scatter plots against B0, time spent inside the test, test success rate, and equalities found and eliminated. For RQ3 we compare calling the test at every full-effort check with the paper's filters (e.g., skipping problems that contain 0/1 variables). The timeout is 120 s (60 s for set (iv)). With about 3,000 instances and 7 configurations, the worst case, where every run times out, is under 600 CPU-hours; in practice it will be far less. The machine used for the tests is yet to be defined.

Soundness checks We will run with cvc5's `--check-models`, use a debug build with assertions, and cross-check every answer against the SMT-LIB status, Z3 and B0 on all sets. Any disagreement is treated as a bug.

4 Milestones and division of work

When	Milestone	Owner
Oct 5–12	Agree on scope with the professor; build cvc5 1.4.1 (debug and release); benchmark scripts; baseline runs (B0, B0$^-$, Z3)	Both
Oct 13–25	Design spike (a)/(b); C1 with model verification	Lara
	Experiment pipeline, *rotate* generator, soundness cross-checks; re-run B1 after every C1 change	Jorge
Oct 26–Nov 8	C2 implied-equality detection; RQ2 benchmark variants	Jorge
	C3 equality elimination (Diophantine solver or own HNF) and mapping back	Lara
Nov 9–13	Progress review: C1 demo, B1 numbers, status of C2–C3	Both
Nov 14–22	B2 runs, RQ3 calling policy and brab ablation, bug fixing	Both
Nov 23–29	Report (LIPIcs) and artifact (README, scripts, raw data)	Both

Each of us reviews the other's code, so that both can explain every part of the project in the interviews.

5 Fallback

If C3 in especial turns out to be too hard, we drop RQ2 and deliver C1 and C2 with a study around it: RQ1 and RQ3 on sets (i), (ii) and (iv), comparing B0, B0$^-$, B1 and B1$^-$ under the different calling policies. The result would, then, measure when the cube test helps in a solver that already has a cheaper relative of it, and whether the cost of indentifying the instances where it would for certain fail (the ones involving equalities) for deciding on applying the test or not does pay off. If the test does not pay off, the report will help understand where and why.

References

[1] B. Dutertre and L. de Moura, "A fast linear-arithmetic solver for DPLL(T)," in *CAV 2006*, ser. LNCS, vol. 4144, 2006, pp. 81–94.

[2] I. Dillig, T. Dillig, and A. Aiken, "Cuts from proofs: A complete and practical technique for solving linear inequalities over integers," *Formal Methods in System Design*, vol. 39, no. 3, pp. 246–260, 2011.

[3] A. Griggio, "A practical approach to satisfiability modulo linear integer arithmetic," *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 8, no. 1–2, pp. 1–27, 2012.

[4] M. Bromberger and C. Weidenbach, "Fast cube tests for LIA constraint solving," in *IJCAR 2016*, ser. LNCS, vol. 9706, 2016, pp. 116–132.

[5] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *TACAS 2008*, ser. LNCS, vol. 4963, 2008, pp. 337–340.

[6] H. Barbosa, C. W. Barrett, M. Brain, et al., "Cvc5: A versatile and industrial-strength SMT solver," in *TACAS 2022*, ser. LNCS, vol. 13243, 2022, pp. 415–442.

[7] C. Barrett, P. Fontaine, and C. Tinelli, *The Satisfiability Modulo Theories Library (SMT-LIB)*, `https://smt-lib.org`, 2016.